

DECLARATION OF WILLIAM ANTHONY MASON
Case IPR2021-0015
U.S. Patent No. 8,380,808

Trials@uspto.gov
571-272-7822

Exhibit No. 2132

UNITED STATES PATENT AND TRADEMARK OFFICE

BEFORE THE PATENT TRIAL AND APPEAL BOARD

MICROSOFT CORP.,
Petitioner,

v.

ROYAL J. O'BRIEN,
Patent Owner.

U.S. Patent No. 8,380,808
Issued: Feb. 19, 2013
Filed: Nov. 24, 2008

Patent Title: DYNAMIC MEDIUM CONTENT STREAMING SYSTEM

Inter Partes Review No. 2021-00015

DECLARATION OF WILLIAM ANTHONY MASON

Contents

I. Introduction 1

II. Claims Construction 6

 A. Person of Ordinary Skill in the Art 6

 B. “extended data”/“attaches extended data to a file stream” 13

 C. “file stream” 16

 D. “data stream” 18

III. Patentability Analysis 20

 A. Claim 1 20

 B. Claim 13 33

IV. Conclusion 39

Declaration 41

I. Introduction

This declaration is a supplement to the declarations that I previously provided in this case (Ex. 2043, Ex. Paper 7) and I hereby incorporate the information set forth in those documents by reference.

I have prepared this declaration to address specific issues raised by the Petitioner in their reply (Ex. Paper 14) and the Reply Declaration of Dr. Houh (Ex. 1035).

Previously, I have observed a number of differences between the prior art teachings and the teachings and claims of the '808 patent. To further assist the Board in reaching a conclusion to this matter, I suggest the most significant difference between the prior art and the '808 patent relates to two specific issues:

1. The '808 Patent teaches a system that requires only a client computer to drive the process, while the prior art references teach a system in which functionality is required to be provided by the server. While the Petitioner has argued that this is not precluded by the prior art, they do so by ignoring the language and figures of the prior art that teach away from this configuration. As I previously noted (Ex. 2043, Page 1) the preferred embodiment of Eylon (Ex. 1005) includes a server (Fig. 1) and further teaches the importance of this by stressing the enhanced performance and security: "As a result, operating system level operations such as copy or erase/delete files, etc., cannot be performed from the client side on the Server file system, further securing the streamed application from unauthorized access and use." (Ex. 1005, Col. 14, Lines 10-13) Eylon distinguishes this as a key distinction of their system: "This is in sharp contrast to other application service providing technologies, which simply extend the file system into the remote server, so that the server looks to the client operating system like a remote disk drive or remote file server." (Ex. 1005, Col. 14, Lines 16-20) While Dr. Houh (Ex. 1035, Page 7, Paragraph 17) extrapolates Eylon's teaching that the server responds to client side demands, he ignores that a system which *only* does so becomes

the very system Eylon eschewed, as it is little more than an extension of “the file system into the remote server.” Thus, there is a significant difference between a server driven data pre-loading system adapting to the failure of its predictive engine to properly anticipate the needs of a client and a system that omits the predictive engine entirely and simply responds to satisfying the on-demand needs of the client.

2. The '808 Patent teaches a system that requires the streaming on-demand mini-filter be present on the client system. The cited prior art systems do not teach a system that requires the mini-filter be present on the client system. File system filter drivers were well-known and described in a 1997 book on Windows file systems development (Ex. 2117, Chapter 12). Eylon would have known about file system filter drivers. Eylon (Ex. 1005) does not include the word filter, as reluctantly admitted by the Petitioner (Ex. 2134, Page 112 Line 8 to Page 114 Line 6). Thus, despite filter drivers being a well-known option to Eylon, he chose not to teach their use. The system of Christiansen (Ex. 1006) teaches us that a “content server” is distinct from the client that is consuming the content (Ex. 1006, Fig. 5). Petitioner claims Christiansen teaches that the minifilter *could* reside on the client: “Patent Owner argues that in Christiansen, the ‘minifilter resides on the server.’ POR at 30. I believe a Skilled Artisan would not agree with that reading of Christiansen because Christiansen states his invention can be implemented on various computer systems, including ‘*personal computers*, server computers, *hand held or laptop devices...set top boxes, programmable consumer electronics, network PCs...*’ Ex. 1006 at [0021] (emphasis added).” The Petitioner’s broad interpretation of Christiansen’s teachings are in error because their interpretation fails to take account to the text of the prior paragraph, which states “The invention is operational...” (Ex. 1006, Para 21) and the fact that the invention includes components *other than* the content servers: “**FIG. 5** is a block diagram representing an exemplary environment in which the present invention may be practiced in accordance with various

aspects of the invention[.]” (Ex. 1006, Paragraph 13) A POSITA reviewing Figure 5 would reasonably conclude that the presence of an internet between the content server and the client is an important element of Christiansen’s teaching because if the cache, including both server and filter, is present on the client, then Christiansen’s system no longer is supporting various web protocols but instead is merely a file system data caching layer. The very benefits Christiansen teaches about having aggregation of currently used content on the content servers are lost: they cannot use streaming web protocols, such as for video or audio, because the client is communicating directly with the storage server. However, Christiansen describes the cache server: “What is needed is a method and system of effectively caching remote data locally such that the entirety of the content set can be stored in separate dedicated storage while a subset of the content is cached on the content server hard disk. Ideally, such a method and system would be mostly or completely transparent to any applications requesting the data.” (Ex. 1006, Paragraph 3). Note that the cache device is “separate **dedicated** storage.” Nothing in Christiansen teaches that the client is dedicated storage. The Petitioner’s reading of Christiansen as teaching such a configuration is incompatible with the actual language of Christiansen.

Christiansen’s background describes the environment in which content servers execute: “A set of content servers in a datacenter, e.g., a server farm, may serve content to many clients at various locations. Previously, the total set of content hosted by a given server farm was relatively small and could be transmitted to and stored on each of the content servers in the farm without excessive costs. Now, however, the amount of content available from a server farm is often in excess of several hundred gigabytes. Buying large capacity hard drives for servers, provisioning them with all the content of a datacenter, and keeping the content on them up-to-date so that they can serve any content requested is expensive both in storage and transmission costs. This is particularly true when more than one datacenter is used to serve the content.” (Ex. 1006, Paragraph 2) In light of Christiansen’s background,

Petitioner's position that Christiansen teaches locating the content sever onto the client device is inconsistent with this environment. Thus, a POSITA would not conclude there was any benefit to using the specific teachings of Christiansen to add the caching server and caching minifilter driver on the client device. Another reason for this is that while Petitioner did control the caching servers shown in Christiansen Figure 5, they would not have controlled the operating system on the clients. Thus, a POSITA would have been motivated not to rely upon the presence of a filter manager because relying upon the presence of a filter manager would have been limited to systems running Petitioner's Windows operating system. Many set top boxes, hand held devices, and programmable consumer electronics ran operating systems other than Windows, none of which had a filter manager. Thus, moving the content server into the client device would not have been possible for several of the demonstrative systems described in Christiansen. Even if it had been possible, the resulting system would have eliminated the benefits of Christiansen's teachings because it means that the content is downloaded to that client device. Downloading to the client device obviates the very benefit Christiansen uses for justifying their invention: "What is needed is a method and system of effectively caching remote data locally such that the entirety of the content set can be stored in **separate dedicated storage** while a **subset of the content is cached on the content server hard disk**. Ideally, such a method and system would be mostly or completely transparent to any applications requesting the data." Downloading a copy to each end platform does not provide the very benefits of caching Christiansen teaches us are so helpful. In other words, when a client begins playing the most recent episode of "Squid Games," the video data can be fetched from a content server that is closer to the client. This in turn means that the data is now closer to *other* clients seeking the same content. Christiansen also teaches that the content server and client are distinct, which is consistent with Figure 5: "The content servers 511-513 may access these objects when providing content to a client. A content server may, for example, host a Web server application that serves

content to clients through networks including the Internet 515. After accessing an object from the file server 505, a content server may then provide the object to a client.” (Ex. 1006, Paragraph 46). A POSITA would not see a benefit to combining the cache server and the client based upon the problem and benefits presented by Christiansen; while Christiansen does not expressly preclude such a combination, they do not provide an example of it, explain any benefits for doing so, or motivate such an implementation.

The presence of the mini-filter *on the client* is an express limitation of the '808 Patent (Ex. 1001, Claims 1, 14): “A streaming on demand system comprising an on-demand requester object installed on a computing device, said on-demand requester object being configured to receive I/O requests on behalf of an application for which data is available in data packs for streaming delivery, said on-demand requester object being responsive to application I/O requests...” I have previously observed that replacing the virtual file system of Eylon with the mini-filter of Christiansen is not an obvious one, the fact that Christiansen teaches the mini-filter is present on the content server creates one more impediment for the POSITA to overcome to find the combination obvious: neither of these prior-art references teach the limitation that the mini-filter must be on the client system.

II. Claims Construction

A. Person of Ordinary Skill in the Art

There is extensive discussion of who the person of ordinary skill in the art is; the Petitioner has argued that their definition is valid because we did not present an alternative. We agreed to accept their definition because we concluded, even with such a remarkable person of ordinary skill in the art the prior art is *still* not obvious.

However, in the Petitioner's most recent filing they created peculiar holes in the memory of their POSITA: this person is knowledgeable about file system mini-filter drivers yet are unfamiliar with basic concepts of file system mini-filter drivers such as the difference between a stream context, stream handle context, and file context, even though these are fundamental concepts of context management used by file system mini-filter drivers on the Windows platform. They criticize my reference to **FltSetStreamContext** because I do not provide an explanation on how to use it, even though this is a function that is fundamental to file system mini-filter development and is thus fundamental understanding I reasonably expected to be obvious to their POSITA and knowing "how the use of the function FltSetStreamContext would attach 'the name and address in the form recorded in the MLAT, offset (location) and length (size)' to a file stream[.]' " (Ex. 1035, Paragraph 9) This is precisely what these context functions do: they associate programmer defined data with a stream. This is *the* quintessential example of associating "extended data" with a file stream in a file system mini-filter.

Specifically, the Petitioner's own documentation describes contexts, how they are used, and that it is the **minifilter** that defines them for a file system mini-filter driver: "At registration time, a minifilter defines the types of contexts it will use, the size of the context and the routine used to cleanup that context. The minifilter uses an array of FLT_CONTEXT_REGISTRATION structures to define these parameters." (Ex. 1010, Page 14) The precise definition of the *contents*

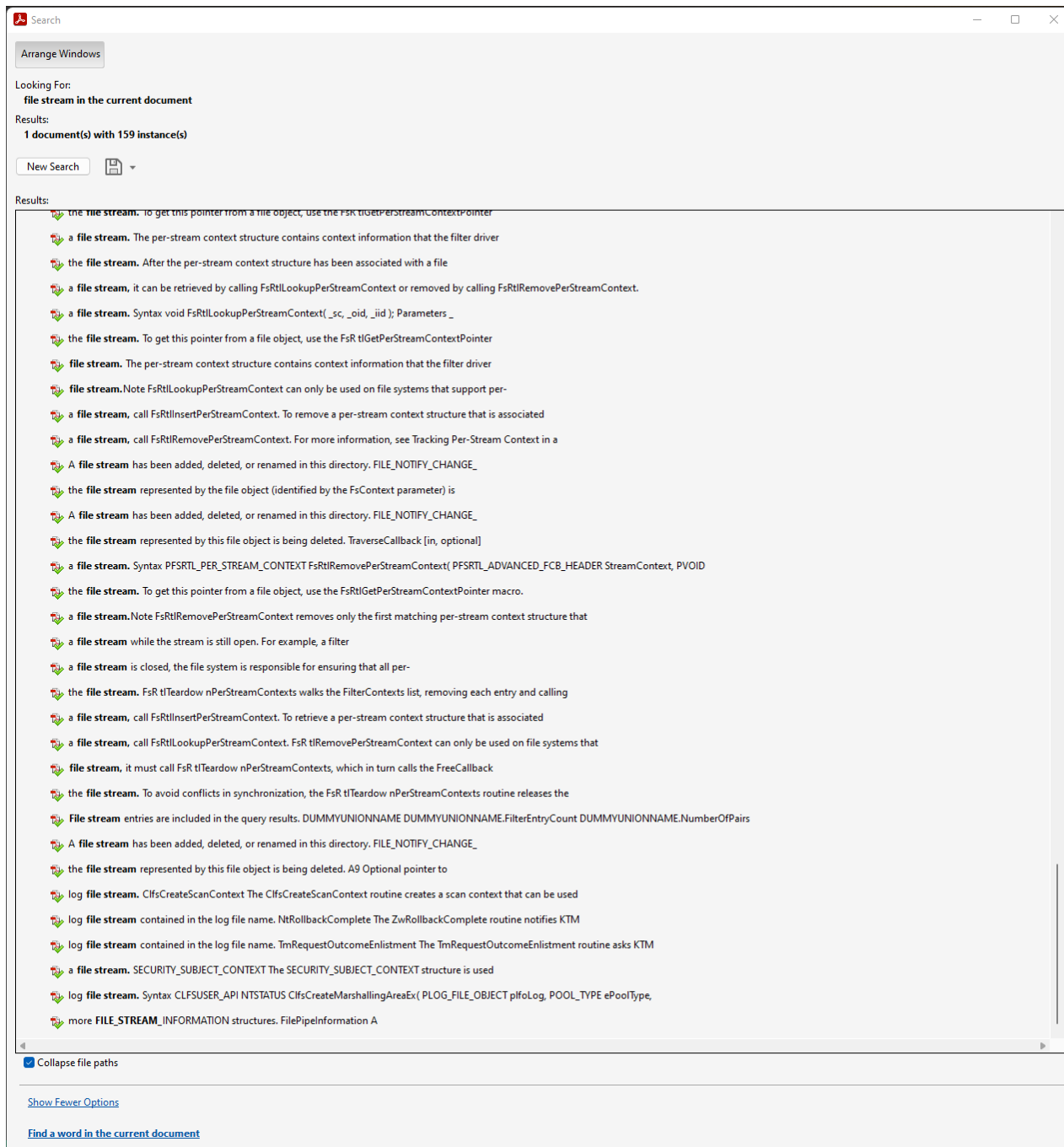
of such a context is up to the developer. Thus, a POSITA, familiar with minifilter contexts, would know that they can store information such as “ ‘the name and address in the form recorded in the MLAT, offset (location) and length (size)’ ” to a file stream[.]’ ” (Ex. 1035, Paragraph 9) That the Petitioner’s POSITA is not familiar with how to use contexts to associate basic information with a stream is not reasonable.

The term “file stream” is a “term of art” that is discussed *extensively* within the Microsoft documentation. To address the Petition’s complaint that I failed to cite a specific page within that document, I note that the term “file stream” is referenced **159** times in the cited document (Ex. 2088). Thus, I did not think it useful to enumerate all of these locations. This is consistent with my position that this is a *fundamental* concept file file system mini-filters, one that is so well described that it would no more be expected to be specified in a patent prosecution history than explaining a screw has threads in a relevant domain. However, the Petitioner’s POSITA was not familiar with this meaning — another example of the “idiot savant” nature of the Petitioner’s POSITA.

The cited reference (Ex. 2088) uses the term “file stream” across a broad range of areas, including filter manager (functions beginning with **F**lt,) file systems such as the Common Log File System (functions beginning with **C**lfs,) file system support functions (functions beginning with **FsRtl** for “file system runtime library,”) and Windows operating systems APIs (data structure declarations using “FILE_STREAM” as part of their name.)

I include a screen capture of a portion of my search for the explicit term “file stream” within the Petitioner’s own documentation (Ex. 2088):

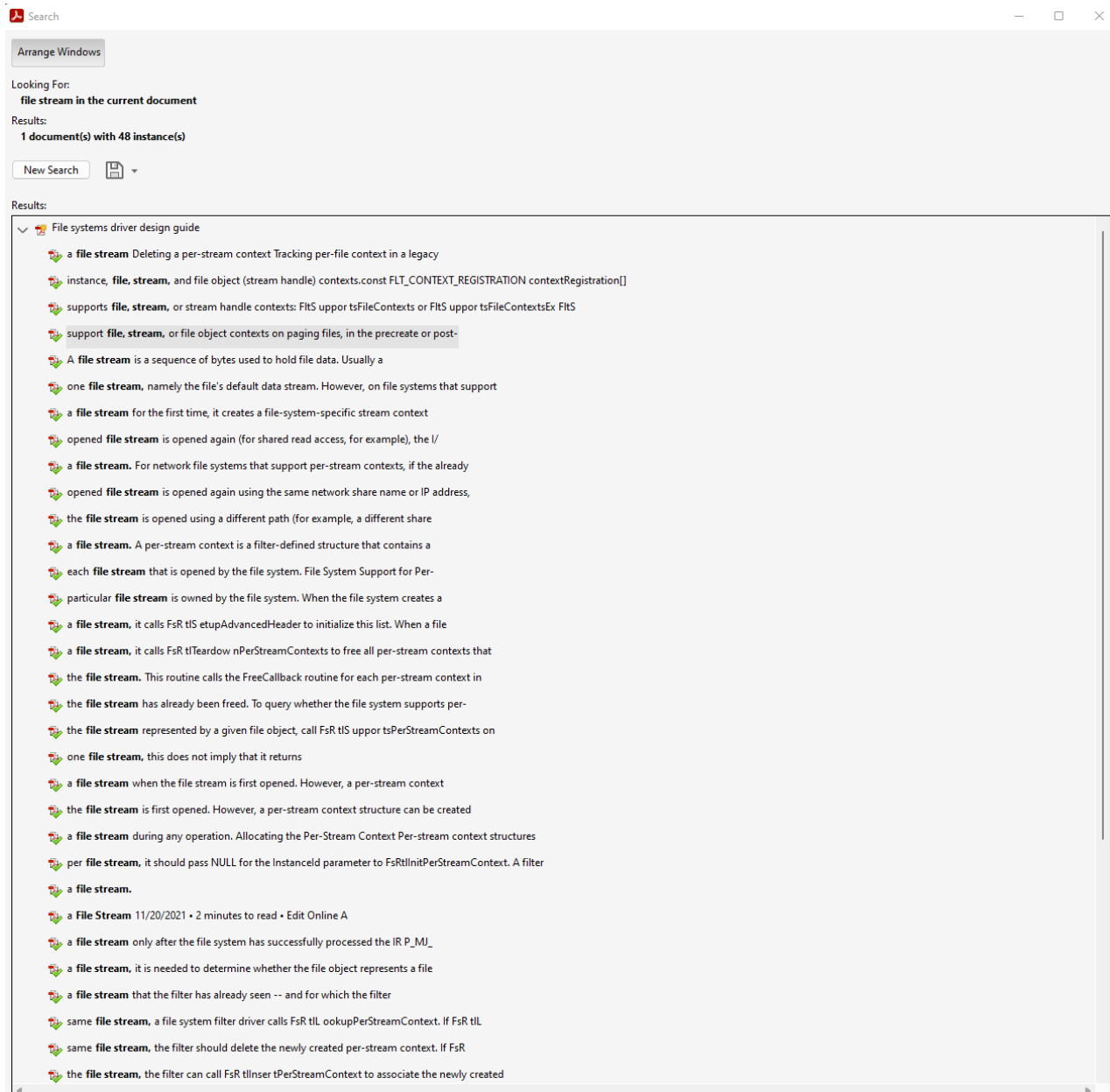
DECLARATION OF WILLIAM ANTHONY MASON
Case IPR2021-0015
U.S. Patent No. 8,380,808



This is the Petitioner’s own document; it is **the** primary reference material that the Petitioner continues to provide today. While it has continued to grow since it was first provided in the early 2000s, the use of the term “file stream” existed prior to 1998 and remains a term of art.

DECLARATION OF WILLIAM ANTHONY MASON
Case IPR2021-0015
U.S. Patent No. 8,380,808

Further, the Petitioner also publishes a current version of their “design guide” from their public facing website. (Ex. 2133) This document provides detailed information about the development of file systems, file system filter drivers, and file system mini-filters:



Note that once again, the term “file stream” occurs 59 times within this document. It is a **core concept** that the Petitioner has been educating developers on how to use for at least 17 years; I was teaching the concept of a “file stream” consistent with its current usage at least by 1996.

The concept and terminology of a “file stream” is the same today as it was in 1998 and 2008. Note that Petitioner’s own early documentation is consistent with the current usage, for example: “**TargetFileObject**: The file object which represents the file stream which this operation affects.” (Ex. 1010, Page 8).

Thus, the Petitioner’s position that a POSITA would think “[T]he ordinary meaning of ‘file stream,’ in the context of the 808 patent, means a transmission of data provided from a file” (Ex. 1035, Page 8) is entirely inconsistent with the Petitioner’s own published information, both current *and* historical.

The Petitioner’s POSITA is carefully crafted: they have the insight to understand combining a server side component (the mini-filter of Christiansen) with a client side component (the VFS and FSD of Eylon) by taking non-obvious cases from each (Christiansen on the client, Eylon’s VFS/FSD converted into a mini-filter that runs on some *other* file system,) a deep knowledge of compression and encryption filters, a background in network data streaming technologies, caching mechanisms, streaming game delivery, streaming application delivery, user mode programming, *and* kernel mode programming — including file systems, file system filter drivers, and file system mini-filter drivers, yet that same POSITA is unfamiliar with specific core concepts.

Thus, while Petitioner’s POSITA definition is entirely unrealistic, a person that does have the requisite skills of this POSITA would *not* reach the conclusions that Petitioner claims that they would reach.

The Petitioner has utilized the opinion of Dr. Houh as an expert with respect to the specific domain of file systems, file system filter drivers, file system minifilters, and on-demand streaming data systems. It the Petitioner’s expert that has set forth Petitioner’s definition of a POSITA. However, Dr. Houh’s level of experience with respect to this area falls short of what I would expect from the Petitioner’s POSITA, let alone qualify as an expert within this domain.

Dr. Houh indicates that his experience in developing kernel drivers is limited to “[A] few device drivers before for network interface cards, and so I have worked in operating system level doing device driver development.” (Ex. 2134, Page 8, Lines 21-24).

Dr. Houh indicates that he has no experience working with dynamically loadable and unloadable driver technologies, though Petitioner’s definition of a POSITA indicates familiarity with these technologies (Ex. 1003 Paragraph 47 and Ex. 2134, Page 12, Lines 1-3).

Dr. Houh is not familiar with the difference between a *device driver* and a *file system driver*. While they do share the common surname “driver” they represent radically different functional implementations and multiple orders of magnitude difference in complexity. (Ex. 2134, Page 10, Lines 11-20)

The primary patent references in this matter are directly related to the Windows operating system. Dr. Houh indicates he has no experience developing kernel mode software on Window (Ex. 2134, Page 17, Lines 7-13). Similarly, he indicates that his only exposure to file systems technologies would have been via the Microsoft Developers Network (MSDN). I note that the tools needed for driver development, including file systems driver development, are not part of MSDN, even today. The current download link for the “Windows Driver Kit” is <https://docs.microsoft.com/en-us/windows-hardware/drivers/download-the-wdk>. It is not clear how Dr. Houh became familiar with these technologies without knowing how to obtain the documentation and tools needed to work with them.

While Dr. Houh does indicate that he has written some device drivers, he also indicated that this was not for the Windows platform. Based upon his own statements, it was for a UNIX platform: “So in this particular device driver that I wrote, the device driver was attached to a specific UNIX device node, and so it handled all particular read write requests for the inode.” (Ex. 2134, Page 10 Line 21 to Page 11 Line 7). I have developed both device drivers and file system drivers for UNIX systems and based upon my own experience, the development of file system drivers is orders of magnitude more complex than device drivers. Similarly, Windows

device drivers are significantly different in form and structure than UNIX device drivers. Finally, I note that Windows device drivers have supported dynamic loading ability since Windows NT 3.1 shipped in August 1993 and thus overlaps with Dr. Houh's own period of study.

Dr. Houh indicates that the Petitioner's POSITA would know how to read the docs to figure out specific details: "Well, I think they would need to be able to read, so that they can read documentation at least..." (Ex. 2134, Page 22 Line 22 to Page 23 Line 10). However, when presented with a reference to a commonly used file system mini-filter function (**FltSetStreamContext**) he was unable to locate it within the existing Microsoft documentation.

Dr. Houh indicates that the Petitioner's POSITA would know how to make a virtual file system, in combination with the teachings of the '808 Patent from their academic background in computer science or related field, and additional experience in a relevant technical field, though he could not articulate what academic classes or experience would be necessary (Ex. 2134, Page 24 Line 1 to Page 28 Line 13).

Dr. Houh indicates he does not have the requisite knowledge to understand the effort required to construct a file system minifilter driver (Ex. 2134, Page 30, Line 15 to Page 32 Line 4.) Further, he indicates that he considers his experience developing one or two non-production network card drivers for some UNIX operating system platform is sufficient ("I did code device drivers before the term 'minifilter' was introduced, and so much of what I have done has some overlap with minifilters and the concept[s] therein." Ex. 2134, Page 30, Line 18-21). Thus, Dr. Houh is applying limited experience with a statically linked network card driver in a UNIX operating system to infer the behavior of filter manager and managed filters, even though filters are much more complex than network card drivers and presently only Windows supports file system minifilter drivers. Dr. Houh has no experience building Windows drivers, file system drivers, file system filter drivers, or file system minifilter drivers, and thus has no basis for the opinions that he offers.

The Petitioner has clearly explained that "file system filter drivers are not device drivers":

File system filter drivers are not device drivers

A *device driver* is a software component that controls a particular hardware I/O device. For example, a DVD storage driver controls a DVD drive.

In contrast, a *file system filter driver* works in conjunction with one or more file systems to manage file I/O operations. These operations include:

- Creating, opening, closing, and enumerating files and directories
- Getting and setting file, directory, and volume information
- Reading and writing file data

In addition, file system filter drivers must support file system-specific features such as caching, locking, sparse files, disk quotas, compression, security, recoverability, reparse points, and volume mount points.

For more details on the similarities and differences between file system filter drivers and device drivers, see the following:

- [How File System Filter Drivers Are Similar to Device Drivers](#)
- [How File System Filter Drivers Are Different from Device Drivers](#)

Source: <https://docs.microsoft.com/en-us/windows-hardware/drivers/ifs/about-file-system-filter-drivers>

Dr. Houh's assertion that his own knowledge of device drivers provides an understanding of filter drivers is contradicted by the Petitioner's own public facing documents.

Based upon his lack of applicable experience with Windows file systems, file system filter drivers, file system mini-filter drivers, and dynamically loadable driver technologies, in my opinion Dr. Houh does not meet the Petitioner's definition of a POSITA.

B. "extended data"/"attaches extended data to a file stream"

I reached my proposed definition of the term "extended data" based upon the use of this term within the context of the '808 Patent. Specifically, the '808 Patent says: "If the I/O request is a create operation, the minifilter attaches extended data to a file stream." (Ex. 1001, Column 3, Lines 11-13) My opinion is that the key to understanding the context of this term and thus its meaning is that the *minifilter* is the actor in this instance. It is my opinion that a POSITA would

understand this in the context of the Petitioner’s own documentation and ordinary usage of this term with respect to a minifilter.

Specifically, a POSITA would understand that the function **FltAllocateContext** would be used to allocate the memory necessary for storing any information the minifilter deems to be useful. The Petitioner pointed this out in their 2004 presentation about how to use filter manager where they introduce “Context Management” and state “Allows minifilters to attach private information to a system object.” Specifically, the Petitioner identifies that context can be associated with a Volume, Instance, Stream, or Stream Handle. The Petitioner also indicates that support for File context is not yet implemented at that time. (Ex. 1011, Page 25)

The Petitioner documents the necessary APIs for attaching this “private information”:

FltAllocateContext see Ex. 1010, Page 14

FltSetStreamContext see Ex. 1010, Page 15. Note that this usage of “stream” is consistent with the specific language used in the ’808 patent, draws from the same specific specialized field of art, and is supported by the Petitioner’s own submissions. The Petitioner, however, argues that “[A] Skilled Artisan (as I previously defined that term) would conclude that Patent Owner’s proposal represents the ordinary meaning of the phrase in the context of the 808 Patent because that function does not appear in the patent or its file history, and there is no evidence that it relates to the system disclosed in that patent.” (Ex. 1035, Paragraph 8) Thus, the Petitioner disclaims the existence of their own previously submitted evidence.

FltDeleteStreamContext see Ex. 1010, Page 16.

FltGetStreamContext see Ex. 1010, Page 17.

FiltReleaseContext see Ex. 1010, Page 18.

It is my opinion that a POSITA would have known the meaning of context to be precisely the type of information that is taught by the '808 Patent: information that is necessary to implement the functionality of the minifilter.

The Petitioner has refined their own documentation over the years, even though the core functionality has not changed. This reflects their decision to more clearly explain critical concepts more thoroughly. For example, the Petitioner states: “The filter manager provides support that allows minifilter drivers to associate contexts with objects to preserve state across I/O operations. Objects that can have contexts include files, volumes, instances, streams, stream handles, and transactions. See About minifilter contexts for minifilter implementation details.” (Ex. 2133, Page 39)

Despite the Petitioner’s own prior submissions and the Petitioner’s own public documentation about the purpose of this minifilter support for contexts, the Petitioner chooses to propose a different construction for this term: “‘associates additional information with a file stream that is the subject of a create operation.” (Ex. 1003, Pages 42-43 and Ex. 1035, Page 4)

The Petitioner’s proposed construction confuses the “extended data” supported by the Petitioner’s filter manager with the specific mechanisms available on Petitioner’s NTFS file system: extended attributes, alternate data streams, and reparse point tags. The Petitioner’s reasoning is flawed.

The proposed construction that I have provided is consistent with the language of the '808 Patent: it captures the idea of associating specific information, such as the MLAT, offset, and length of the data packs which make up the file. In addition, this is the same process shown in Figure 4 of the '808 Patent, where at step 422 it shows “Attach Extended data to File Stream” corresponding to the use of **FiltAttachStreamFileContext** (Ex. 1010, Page 15 and Ex. 2133, Page 192). Similarly the '808 Patent Figure 4 shows retrieval of this extended data in step 404

“Does extended data exist?” corresponding to the use of **FltGetStreamFileContext** (Ex. 1010, Page 17 and Ex. 2133, Page 193).

Thus, it is my opinion that the correct construction for the term “extended data” is “data unrelated to the contents of the file stream associated with the file stream by a file system, file system filter driver, or file system mini-filter driver.” (Ex. 2043, Pages 13-15) A POSITA would have understood this meaning because it is consistent with the Petitioner’s own current documentation, which is consistent with the concepts and usage since at least the original release of the Petitioner’s filter manager. Further a POSITA would have understood that “attaches extended data to a file stream” is realized by using the very mechanism that Petitioner provides and documents to achieve this purpose.

In addition, my proposed construction does not include the NTFS features that would be inappropriately included by using the Petitioner’s definition; nothing in the teachings of the ’808 Patent rely upon, or even suggest using, these NTFS features.

C. “file stream”

As I noted in § A. the term “file stream” is a term of art with respect to Windows file systems; this term of art is embedded within the file system, file system filter, and file system mini-filter model present in Windows. As I noted, the term “file stream” is used numerous times throughout the cited document (Ex. 2088). It is such a basic concept, invented by Petitioner, that I did not expect them to so materially deviate from their own documentation. The Petitioner’s POSITA would be familiar with this material, because this is fundamental information for anyone constructing file systems, file system filters, and/or file system mini-filters in the Windows operating system environment.

Petitioner explains the meaning of the term “file stream” through its own public facing documentation: “A file stream is a sequence of bytes used to hold file data. Usually a file has only one

file stream, namely the file's default data stream. However, on file systems that support multiple data streams, each file can have multiple file streams. One of these is the default data stream, which is unnamed. The others are named alternate data streams. When you open a file, you are actually opening a stream of the given file." (Ex. 2133, Page 283)

Similarly, the Petitioner explains: "When a file system opens a file stream for the first time, it creates a file-system-specific stream context structure, such as a file control block (FCB) or stream control block (SCB), and stores the address of this structure in the FsContext member of the resulting file object.

"For local file systems, if the already opened file stream is opened again (for shared read access, for example), the I/O subsystem creates another file object, but the file system does not create a new stream context. Both file objects receive the address of the same stream context structure. Thus, for local file systems, the stream context pointer uniquely identifies a file stream." (Ex. 2133, Page 283)

I have personally implemented file systems that support "per stream" contexts using the documented functions (e.g., "FsRtlInitPerStreamContext" Ex. 2088, Page 1782), legacy file system filter drivers that rely upon this support to retrieve stream contexts (e.g., "FsRtlLookupPerStreamContext" Ex. 2088, Page 1841,) and file system mini-filter drivers that rely upon the Filter Manager APIs to managed stream contexts (Ex. 2133, Pages 39-40, 185-199). My proposed construction, based upon my expertise in this domain is: "[A]n open instance of a file or, for a file system that permits multiple data streams within a given file or directory, an open instance of the stream." (Ex. 2043, Page 18)

Petitioner proposed a substantially different construction to the Board than they have previously provided to the general public: "[T]he ordinary meaning of 'file stream,' in the context of the 808 patent, means a transmission of data provided from a file." (Ex. 1035, Page 8)

In fact, I suggest a less limited construction is more consistent with Petitioner's previously published definition of this term: "An instance of a file or, or a file system that permits multiple

data streams within a given file or directory, an instance of a specific stream.” This definition is consistent with Petitioner’s own published definition, but differs from the Petitioner’s proposed construction in this specific case. This definition is also consistent with the usage within the ’808 Patent, which uses “file stream” according to its usual meaning within the art.

D. “data stream”

The Petitioner confuses the specific term “data stream,” which is used within the ’808 patent, with more general terms. As I pointed out in my previous declaration (Ex. 2043, Pages 15-17) the term is used in a broad range of areas, each with subtly different meanings. The Petitioner rejects my description that its use within the ’808 patent is similar to the way it is used in the Java programming language (e.g., as a means of encoding.) However, the petitioner fails to demonstrate why my proposed construction is insufficient or incorrect.

For example the Petitioner maintains that “the term ‘data stream’ should be no narrower than ‘file stream’ as ‘data’ is a broader concept than ‘file.’ ” (Ex. 1035, Page 9, Paragraph 14) However, this conflates the contextual use of the term “data stream” within the ’808 Patent, because the term “data stream” is described as consisting of a set of “data packs” (Ex. 1001, Claims 1 & 14). In turn, the “data pack” is distinguished because it has a unique offset, length and location: “...configured to reference a table that includes names and paths with at least one offset address and length where each data pack required to fulfill each I/O request is located...” (Ex. 1001, Claims 1 & 14). The very purpose of the “data packs” is to describe the data source for some *portion* of the “data stream.” Despite this, the Petitioner claims “[A POSITA] would conclude that the ordinary meaning of ‘data stream’ is ‘data provided from a data source.’ ” This proposed construction discards the explicitly described purpose of “data packs.” That is why I proposed using the Java definition of a “data stream” because it captures the *structure* of the underlying data, which for the ’808 Patent includes the location(s) where the given data pack is located. The Petitioner’s

DECLARATION OF WILLIAM ANTHONY MASON

Case IPR2021-0015

U.S. Patent No. 8,380,808

definition would limit “data streams” to a single storage location, which is incompatible with the language of the ’808 Patent and thus incorrect.

III. Patentability Analysis

A. Claim 1

The Petitioner takes the naïve position that there is a direct correspondence between the application initiated read and the operating system initiated retrieval of data. While this level of description is fine for providing a simple model to a new student, it is certainly not the level of understanding of the Petitioner’s POSITA, as the Petitioner’s POSTIA is familiar with caching, which is present in both Eylon (Ex. 1005, Colum 4, Lines 10-22) and Christiansen (Ex. 1006, Paragraph 3). All modern operating systems cache data, which dramatically improves performance. While I have explained the actual context in which the systems that both Eylon and Christiansen would have operated, the Petitioner’s position is to jettison decades of their own teachings in favor of their simple narrative. While I disagree with their analysis, as I have previously explained, I would suggest that the key distinction is that both Eylon and Christiansen teach “server centric” systems, while the ’808 Patent teaches a “client centric” system.

The Petitioner’s position is that Eylon teaches a system in which the server side functionality can be omitted (Ex. 1035, Paragraph 17). While Eylon uses client-side demand to *start* this transfer, it is used either as a trigger event (Ex. 1005, Figure 5), or as an indicator that the data is not in the cache (Ex. 1005, Figure 6A), likely due to a failure of the pre-fetch logic: “When the user strays from the predictive path and an on-demand request is issued, the predictive routine can be re-centered on the appropriate node associated with the on-demand request and the predictive stream restarted from that point.” (Ex. 1005, Column 7, Lines 4-8)

Note that the Petitioner also fails to address the explicit functionality that Eylon ascribes exclusively to the server in their system: “Advantageously, the present architecture described enables a positive user experience with streamed applications without requiring constant use of broadband data links. In addition, and unlike remote-access application systems, the streaming

applications execute on the client machine and server and network resources are utilized for delivery only. As a result, server and network load are reduced, allowing for very high scalability of the server-based application delivery service which can deliver, update and distribute applications to a very large number of users, as well as perform centralized version updates, billing, and other management operations. Furthermore, because the application server does not run application code but only delivers streamlets to the client, it can run on operating systems other than the application target operating system, such as Unix derivatives or other future operating systems, as well as Windows-based servers. In addition, the application streaming system is source code independent. Applications are packaged automatically on the server in accordance with an analysis of the sequence in which the application is loaded into memory during execution. No changes to the application itself are required.” (Ex. 1005, Col. 4 Line 51 through Col. 5 Line 5) I previously discussed these important features of Eylon’s server-oriented system (Ex. 2043, Page 41). Even if the Board were to accept the Petitioner’s argument that the predictive streaming components could be omitted, Eylon’s system still requires a server.

While Petitioner’s position is that Eylon does not teach away from a system without a predictive path, Eylon teaches that a system with the predictive paths is clearly superior to one without:

- “Preferably, the server is configured to automatically forward sequences of streamlets to the client using a predictive 45 streaming engine which selects streamlets to forward according to dynamic statistical knowledge base generated by analyzing the various sequences in which the application program attempts to load itself into memory as various program features are accessed.” (Ex. 1005, Column 6, Lines 43-49)
- “Additional application streamlets can then be actively pushed to the client in accordance with the predictive model and possibly additional data forwarded to the server from the client while the application executes.” (Ex. 1005, Column 4, Lines 45-58)

- “In many cases, as a result of the predictive streaming, the desired page will already reside in the Cache/VFS 160.” (Ex. 1005, Column 10, Lines 47-49)
- “In particular, the server can reset position in the Usage Data graph 172 to reflect the user’s new position in the application’s usage space to ensure that the predictive algorithm will continue to predict the most likely streamlets operating systems. In this example, the Startup Block contains 28 k of data in seven 4 k streamlets, six of which are associated with various sections of the executable, and one which is associated with the DLL (and here is the DLL header). The streamlet map database 200 indicates which in response to user’s actions.” (Ex. 1005, Column 11, Lines 9-13)
- “In some implementations, the server 12 can keep track of the streamlets which are present on a given client and use this information as part of the predictive streaming process.” (Ex. 1005, Column 13, Lines 15-18)
- “In response, the server can use the predictive streaming routines to identify those streamlets which are present on the client and which are least likely to be needed and forward this information to the client so the identified streamlets can be purged.” (Ex. 1005, Column 13, Lines 25-29)

Thus, a POSITA would learn from Eylon that the server-side system is required for important functionality, and the omission of the predictive streaming technology from that implementation would be materially penalized.

Eylon also teaches us that the server is an essential part of this optimization **because** it combines the feedback from multiple clients: “The predictive graph of usage probabilities can be updated continuously in response to the usage data provided by the clients connected to the server.” (Ex. 1005, Column 7, Lines 9-11) Eylon similarly teaches us that this server-side predictive

model is generally beneficial: “Such information can be useful in refining a predictive model used to determine an optimal order in which to send streamlets to a client.” (Ex. 1005, Column 13, Lines 8-11)

Finally, Eylon distinguishes their teachings from prior art work where the client drives the download of data: “This is in sharp contrast to other application service providing technologies, which simply extend the file system into the remote server, so that the server looks to the client operating system like a remote disk drive or remote file server. Such schemes can are also be substantially slower, since they do not take advantage of ‘deep’ predictive algorithms, but can only use simple, heuristic approaches such as cache read-ahead, which deliver several contiguous blocks of data into the local Client context in addition to the required block(s).” (Ex. 1005, Column 14, Lines 16-25) Removing the server side system that Eylon teaches, including the highly beneficial predictive streaming that Eylon teaches creates a system where the client fetches data from the server as driven entirely by the logic of the client itself. This is, however, the very system of a “file system into the remote server” that Eylon criticizes. Thus, a POSITA would understand that Eylon teaches away from a system with only client-side logic, which is a distinguishing characteristic of the '808 Patent's limitations.

The Petitioner has admitted that Eylon teaches a system that is more than a remote file system (Ex. 2134, Page 100): “What they [remote file systems] do is they go to a remote drive and then cache in the whole file.”

Interesting, the Petitioner then also establishes a position that it is the use of “streamlets” that also distinguishes Eylon from prior work: “What they [remote file systems] don't do is part of what Eylon's disclosing, which Eylon allows these things called streamlets in the file to bring in only portions of files that are requested. These previous file systems, like AFS and NFS, they didn't do that. They either cached the whole file or didn't cache the whole file.” (Ex. 2134, Page 100 Lines 7-13) This statement is false. I know this because one of my very first responsibilities at Transarc when I started in 1989 was to *modify* the data caching component to use smaller

caching units. When I completed my work, AFS 3 supported variable sized cache units (called “chunks”) that were used to cache *portions* of files, and openAFS, the direct successor to AFS 3 and using the same code base, which is freely available still supports variable sized “chunks”. Indeed, it is my own prior experience and understanding of what makes sense in such a remote file system that helped me understand Eylon’s use of “streamlet” was the same as the AFS concept of a “chunk”. In addition, AFS (in 1989) supported applications that ran even if the entire contents of all of the files needed to execute the application did not run. Subsequent to my time working with AFS I was involved in the development of DCE/DFS, which was considered the successor to AFS. It also had the ability to dynamically stream data from remote servers to local servers and was available as a commercial product in the early 1990s. The Open Software Foundation (now the Open Group, responsible for UNIX operating systems standards) made the source code for all these components publicly available in 1997 and it includes support for on-demand transfer of data (in integer multiples of fixed-sized *pages*.) Caching was not done on a whole file basis, though data transfer was done in terms of fixed sized units. Thus, based upon my personal experience, Petitioner’s position is false. Further, based upon the Petitioner’s definition of a POSITA, it is clear that the POSITA would have known about these prior art systems, given that they were actively used, published, and well-known in the field of file systems.

While Eylon does teach using on-demand download, it is done in tandem with the server’s predictive download and the benefits Eylon teaches us are not achieved if we remove the server’s predictive download. In other words, to reach the Petitioner’s position that predictive streaming is optional: “While Eylon does disclose predictive streaming, Eylon makes clear that his predictive streaming is an optional functionality” (Ex. 1035, Paragraph 17) Petitioner ignores that it removes all of the benefits of the teachings of Eylon.

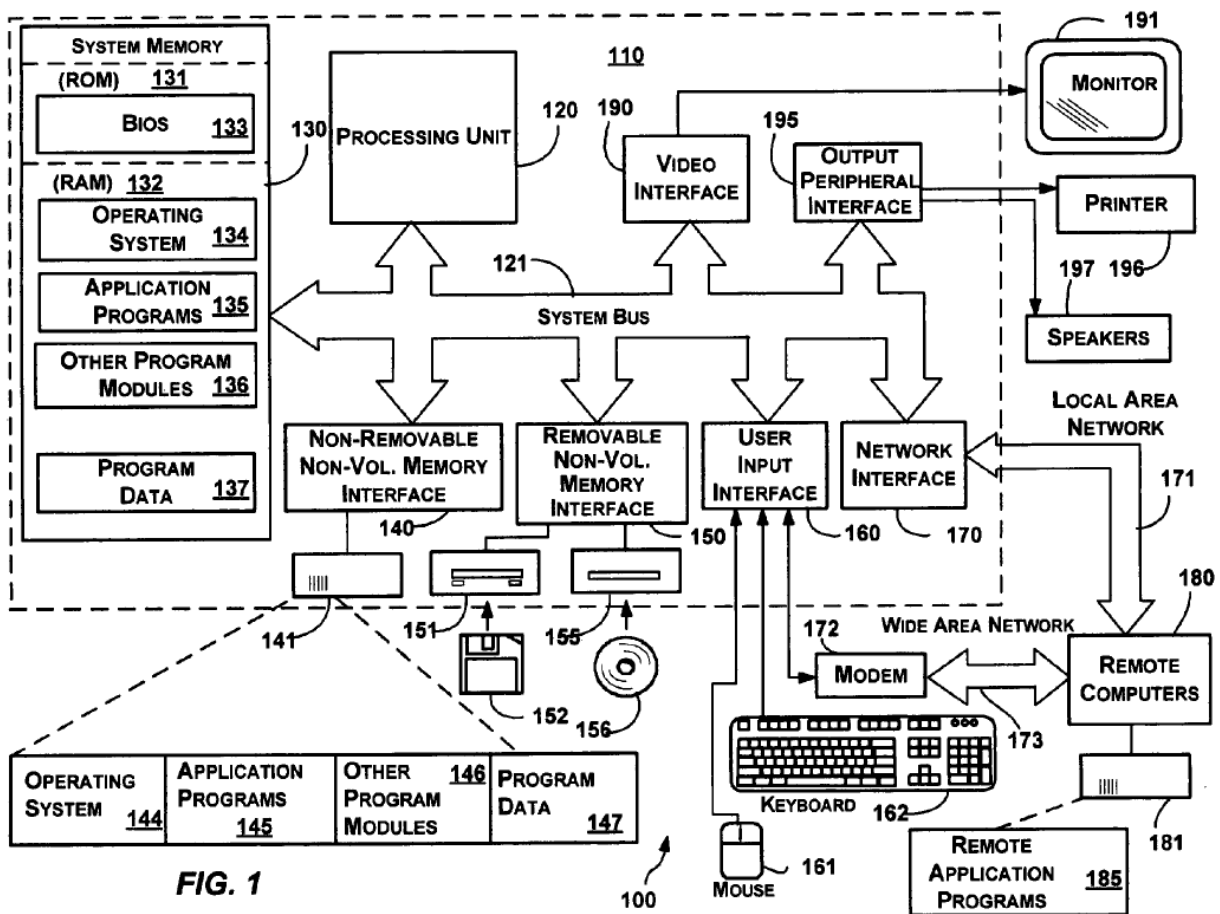
Christiansen does not overcome this server-focused implementation model, either. The Petitioner has argued that Christiansen does *not* teach that the mini-filter resides on the server. However, the Petitioner is being disingenuous by suggesting that Christiansen expects to find the

“content server” on “ ‘**personal computers**, server computers, **hand held** or **laptop devices**, **set top boxes**, **programmable consumer electronics**, **network PCs...**’ Ex. 1006 at [0021] (emphasis added).” (Ex. 1035, Paragraph 22).

The full text of Paragraph 21, however, makes it clear that “the invention” of Christiansen is not *just* the computer on which the content server resides, but also the computers that consume said content: “The invention is operational with numerous other general purpose or special purpose computing system environments or configurations. Examples of well known computing systems, environments, and/or configurations that may be suitable for use with the invention include, but are not limited to, personal computers, server computers, handheld or laptop devices, multiprocessor systems, microcontroller- based systems, set top boxes, programmable consumer electronics, network PCs, minicomputers, mainframe computers, distributed computing environments that include any of the above systems or devices, and the like.” (Ex. 1006, Paragraph 21)

However, to understand the meaning of “the invention” we need to turn to the prior paragraph: “FIG. 1 illustrates an example of a suitable computing system environment 100 on which the invention may be implemented. The computing system environment 100 is only one example of a suitable computing environment and is not intended to suggest any limitation as to the scope of use or functionality of the invention. Neither should the computing environment 100 be interpreted as having any dependency or requirement relating to any one or combination of components illustrated in the exemplary operating environment 100.” (Ex. 1006, Paragraph 20)

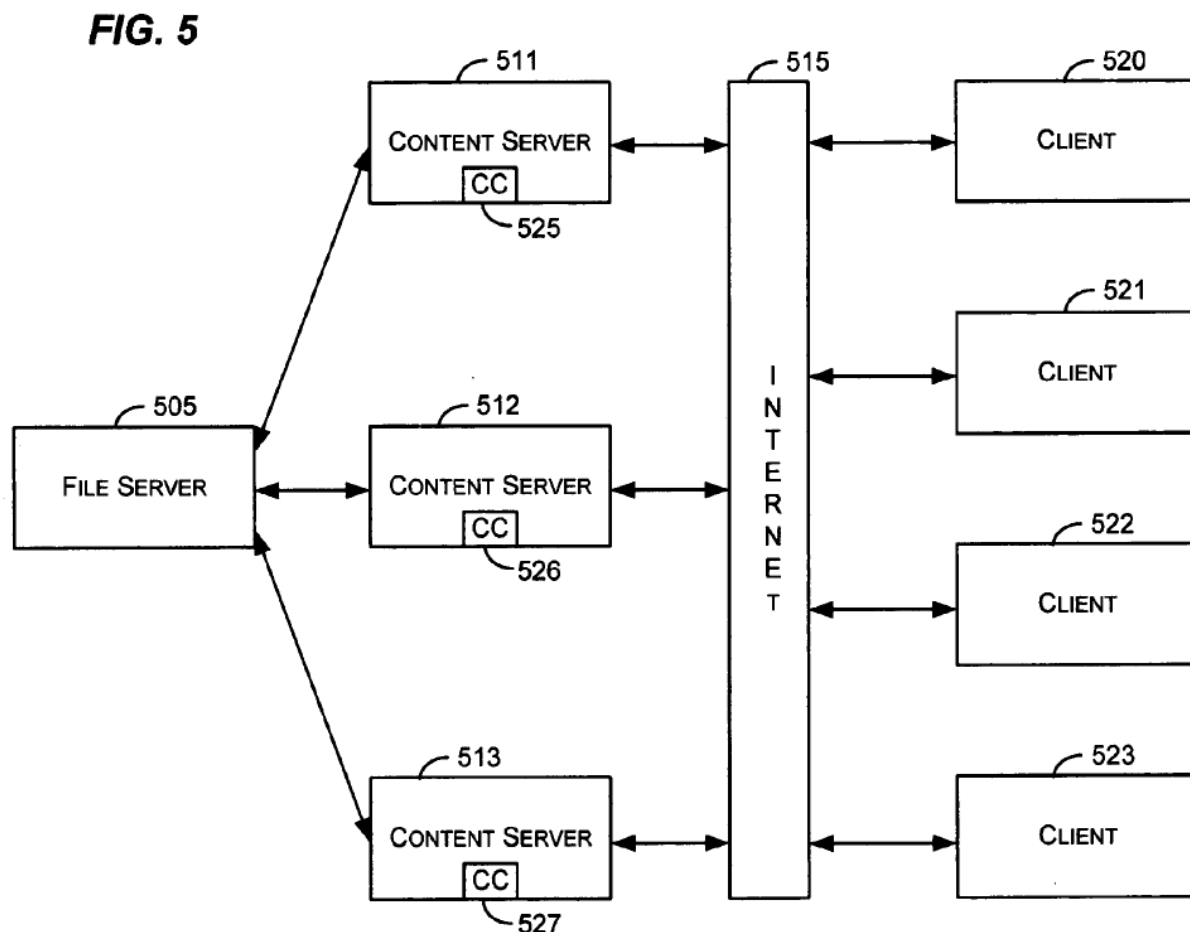
Christiansen’s Figure 1 is nothing more than a rather generic image of a potential computer. While this makes sense, given that computer architecture and design is largely standardized, it only provides a broad context in which to understand Christiansen’s teachings:



Ex. 1006, Figure 1

Server computers typically, particularly of the type described by Christiansen “A set of content servers in a datacenter, e.g., a server farm, may serve content to many clients at various locations.” (Ex. 1006, Paragraph 2). Such data center servers do not have keyboards, mice, displays, speakers, printers, or keyboards, though such data center servers *could* have such items. The devices the Petitioner emphasized in their own text (Ex. 1035, Paragraph 22) are in fact the types of devices one would use to consume content.

Without content consumers, there is no need for content servers. Without content servers, there is no need to store content and thus no need for file servers. Similarly, without content servers, there can be no content consumers. This is more clearly shown in Christiansen Figure 5:



Ex. 1006, Figure 5

This configuration is highly consistent with Christiansen's specification: content stored within a data center, which then provides content to end user devices that consume said content. Figure 5, in fact, also fits with the text the Petitioner quotes from Paragraph 21: "clients" must be present in order for Christiansen's invention to have actual purpose.

Thus, the Petitioner takes the position that these client computers *could* have the content filter present on them (Ex. 2134, Page 130 Line 11 to Page 131 Line 6.) because they are included as part of Figure 5. The Petitioner, however, cannot point to any text within Christiansen that supports this interpretation of Christiansen (Ex. 2134, Page 131, Lines 11-25).

Much of Christiansen teaches how filters work (Ex. 1006 Paragraphs 30-44). These descriptions are not specific to the “caching server” that provides Christiansen’s claimed benefits. It is clear from Christiansen’s text that this functionality is general purpose in nature, and not specific to their claimed caching server benefits: “For example, antivirus products provide a filter that watches I/O to and from certain file types (.exe, .doc, and the like) looking for virus signatures, while file replication products perform file system-level mirroring. Other types of file system filter drivers are directed to system restoration (which backs up system files when changes are about to be made so that the user can return to the original state), disk quota enforcement, backup of open files, undeletion of deleted files, encryption of files, and so forth. Thus, by installing file system filter drivers, computer users can select the file system features they want and need, in a manner that enables upgrades, replacement, insertion, and removal of the components without changing the actual operating system or file system driver code.” (Ex. 1006, Paragraph 31) While such filters might exist on client computers, they are unrelated to the specific implementation of a “caching server.”

The section entitled “Caching Remote Content Locally” is where Christiansen begins to discuss the specific filter implementation they use on the content server to provide the benefits of Christiansen’s teachings. Note that this section, which is a description of the content server, refers to Figures 5-11. Figure 5 shows the “content server” and “cc” (presumably, content cache) as distinct from the “file server” (presumably where the content files are stored) and the “clients” (presumably where the content files are consumed). I note that this diagram indicates the content server and the client are separated by the “internet” (515). This suggests that the content server is not resident on the client.

Thus, Petitioner’s argument that Christiansen does not exclude the filter being on the client would reasonably infer that the content server, which is bound to the minifilter implementation Christiansen taught, must be resident on the client in said configuration. However, Christiansen does not explain how this would be done and the Petitioner does not explore this combination.

If the cache server is present on the client — those systems Petitioner highlighted in Ex. 1035, Paragraph 22, then there is no benefit to the proposed system, since it becomes a system in which each client simply downloads content from the file servers. This is exactly the system that Christiansen teaches is the problem: “A set of content servers in a datacenter, e.g., a server farm, may serve content to many clients at various locations. Previously, the total set of content hosted by a given server farm was relatively small and could be transmitted to and stored on each of the content servers in the farm without excessive costs. Now, however, the amount of content available from a server farm is often in excess of several hundred gigabytes. Buying large capacity hard drives for servers, provisioning them with all the content of a datacenter, and keeping the content on them up-to-date so that they can serve any content requested is expensive both in storage and transmission costs. This is particularly true when more than one datacenter is used to serve the content.” (Ex. 1006, Paragraph 2)

The configuration that Petitioner suggests Christiansen teaches does not address the original problem identified by Christiansen. Indeed, that is the very system that existed prior to Christiansen. Thus, there is nothing in Christiansen that would lead a POSITA to understand that the content server, including the mini-filter, be stored on the client device. There is no benefit with respect to the teaching of Christiansen, it does not solve the problem identified by Christiansen, and thus is not a realistic interpretation of Christiansen.

The requirement that the logic be entirely present on the client is an explicit limitation of the '808 Patent's teachings and claims. Thus, the claims of the '808 Patent are not obvious over the teachings of Eylon and Christiansen, because they are both server side technologies. Considering Eylon without server side predictive streaming yields a system that is indistinguishable from Christiansen without content servers between storage and client: they look like network storage.

The Petitioner interprets the '808 Patent to imply that application I/O and Paging I/O are equivalent because the '808 Patent describes them each in an explanation of the '808 Patent's

Figure 4. However, this interpretation is *not* explicit in the text, nor would a POSITA understand them to be the same, as I have previously explained at length (Ex. 2043, Page 19-37). The Petitioner is taking a complex drawing and attempting to obfuscate the differences that would be known to a POSITA since such a person would have worked with file systems, file system filter drivers, and file system mini-filter drivers. Since the Petitioner's POSITA is familiar with these technologies and the operating systems in which they operate, they would understand that Paging I/O cannot be initiated directly by an application. The I/O system of the kernel must handle both application I/O and Paging I/O.

The Petitioner is well aware of this distinction. The Petitioner has published the source code for the FAT file system for roughly 20 years. It is presently located online at <https://github.com/microsoft/Windows-driver-samples/tree/master/filesys/fastfat>. While this code has changed slightly over the years it has been available, the core implementation for the read functionality has not. In the current version of **FatCommonRead**, the function in the FAT file system that handles *all* I/O operations given to the file system the code starts at line 518 and ends at line 1688 (read.c at the URL I specified earlier in this paragraph.) That is, it represents over one thousand lines of code. No patent filing is ever going to capture the complexity and nuance, but this is exactly the behavior that the Petitioner's POSITA would know, because this is the published example file system source code.

Instead of demonstrating how an application program can directly generate a paging I/O — which is not possible using the existing Windows APIs — the Petitioner instead attempts to take a complex diagram and 37 lines of text to assert that paging I/O and application I/O are the same. I have built file systems and file system filter drivers. I know from my direct personal experience that a file system must support all forms of I/O. The Windows filter manager also handles all forms of I/O; it passes only those forms of I/O requested by the mini-filter. Despite the fact this is the Petitioner's own system, they now assert that it behave other than how their own code samples are implemented, their own documentation describes.

Petitioner has taken a position that is contrary to their own extensive documentation, their code samples, the general understanding of such concepts to those that work with file systems, file system filter drivers, and file system minifilters.

Note that Petitioner does not call upon any of their own subject matter experts in this area to disagree with my conclusions. These are the people who designed, developed, enhanced, and documented how filter manager work. I know, based upon personal knowledge, that a number of those people still work for Petitioner. Such domain experts would easily be able to identify any flaws in my own analysis and provide authoritatively correct information. Instead, Petitioner relies upon an outside expert with no experience building file systems, file system filter drivers, or file system minifilter drivers and is not even able to clearly defined the difference between “user mode” and “kernel mode” (Ex. 2134, Page 73 Line 11 to Page 76 Line 18). This **is** a question that I would expect a POSITA to be able to answer because it is a fundamental concept that we teach to anyone taking an operating systems course.

The Petitioner suggests that the table of the '808 Patent, Claim 12 could be augmented by the “logged entries of Prahlaad.” It is my opinion that nothing in Prahlaad teaches this sort of association of log entries with individual files; this is the basis of my original objection to this reference because Prahlaad teaches a fairly standard mechanism for tracking changes in a fashion that can be applied to a remote system suitable for replicating the information. Converting the temporal ordering that is essential to correct replay in Prahlaad’s system into a per-file ordering would sacrifice the consistency that is at the heart of Prahlaad. Note that the word “consistency” is used 87 times within Prahlaad; this idea of Consistency is not a mere curiosity to Prahlaad but is core to Prahlaad: “the consistency point marker being indicative of a known good state of the computer application” (Ex. 1016, Col. 5, Lines 16-18)

I have implemented replicated data systems similar to that described by Prahlaad. Indeed, one of my earliest Windows file system filter drivers (mid-1990s) was used by IBM as part of a data replication system used in point-of-sale, including the United States Post Office. I have

implemented strongly consistent file systems, such as the Episode file system that I co-developed at Transarc in the 1990s. I was responsible for developing the transactional log system, the key component in restoring consistency to the file system meta-data after an anomalous event such as as system crash or power outage. I would note that the IBM product was in use for more than 20 years and the Episode file system is still in use today as part of IBM's z/OS operating system.

A key requirement for being able to ensure data consistency is to preserve the temporal order of operations. In other words, the records in the log need to capture the sequence of events on the original system so that they can be replayed in the same order on the target system. This is how we maintain consistency between replicas.

Thus, my objection to the Petitioner's proposed combination of Prahlad with the previously cited prior-art works: a POSITA would not be motivated to combine these references because Prahlad teaches a temporally ordered log that combines I/O activities to numerous files into a single source. Further, the system taught by Prahlad cannot provide consistency if it is converted to use a per-file stream model because it would lose the cross-file temporal ordering.

This temporal ordering is not a concern for Eylon, Christiansen, *or* the '808 Patent. The primary emphasis in all three is to efficiently cache static content on consumer devices (Eylon, '808 Patent) or intermediate content caching servers (Christiansen). While Christiansen does suggest that it could be used to migrate changes from content servers to file servers, it is not a core teaching and Christiansen does not even mention the issue of cross-file consistency that is at the heart of Prahlad.

Thus, it is my opinion that a POSITA would have no motivation to combine Prahlad with Eylon and Christiansen to implement a per-file log system that is somehow equivalent of the MLAT taught by the '808 Patent.

B. Claim 13

The Petition argues that the motivation to combine Eylon with Christiansen is to combine the streaming functionality of Eylon on the system of Christiansen (Ex. 1035, Page 15). This proposed combination fails to be properly motivated however:

- Eylon already implements the streaming functionality of the proposed combination without using the minifilter of Christiansen. The Petitioner’s argument to combine them is that minifilters are easier to implement than filters as I have previously noted: “[Petitioner’s] advantages are entirely the advantages of implementing a file system mini-filter rather than a file system filter driver.” and “[Petitioner] fails to explain the advantages of implementing the FSD and VFS of Eylon as a file system filter drive.” (Ex. 2043, Page 54) Absent evidence that there is some benefit to implement Eylon as a filter driver, there is no benefit of combining Eylon with Christiansen in any case.
- If we remove the predictive streaming of Eylon that the Petitioner has argued is not required (Ex. 1035, Paragraph 17) the functionality of Christiansen’s system is equivalent to that of Eylon. Recall that Christiansen is simply supporting existing mechanisms for delivering content to the client computers. Christiansen Figure 5 indicates the “internet” that separates the content server from the client. Christiansen in fact suggests using a “web server application” which would usually mean using the HTTP protocol. Windows XP (2002) included the WebDAV client, which provided a data streaming protocol based file system for transferring data to clients. I note there were other streaming data protocols as well that would have worked with Christiansen’s web server application as well. The Petitioner fails to motivate why a POSITA would need the streaming technology of Eylon.

- Real-world experience indicates that the proposed combination would be more complicated than the original system of Eylon as I have previously noted: “A POSITA would not have been able to construct a file system filter or mini-filter that implemented the functionality of Eylon successfully.” (Ex. 2043, Page 55) Despite the fact that the Petitioner has subject matter experts in their employ with practical experience implementing file systems, file system filter drivers, and file system mini-filter drivers on the Petitioner’s own operating system, using the Petitioner’s own filter manager, the Petitioner has not presented evidence to rebut my opinion.

Thus, it remains my opinion that a POSITA would not have been motivated to combine Eylon and Christiansen in the way proposed by the Petitioner. Further, it is my opinion, based upon my direct experience, that had Petitioner convinced the POSITA to do so, the POSITA would have failed absent intervention by a suitably skilled expert, such as those in the employ of the Petitioner.

The Petitioner argues that Eylon teaches the handling of both application and paging I/O requests and does so by citing Eylon (Ex. 1005, Column 4, Lines 15-22). However, the Petitioner fails to capture the full context of that paragraph: “During normal operation of an application, the Windows operating system will periodically generate page faults and associated paging requests to load portions of the application into memory. Standard operating system procedures direct these I/O requests to the proper data device driver for processing. When the streaming application is initialized, the operating system is informed that the relevant data is stored on the VFS drive. Thus, as the streaming application executes and paging or data I/O requests are generated to retrieve require code or data, the operating system will automatically direct it to the FSD driver which then passes it in the proper format to the VFS caching system for processing.” (Ex. 1005, Column 4, Lines 10-22)

It seems Petitioner implies that “data I/O” and “application I/O” are synonymous terms within the context of this text, but the only I/O that would normally be generated are paging I/O operations, even for data, as I previously explained (Ex. 2043, Page 55-56).

I previously noted that the Petitioner makes the source code to versions of the FastFat file system available. The delineation between cached and non-cached I/O is clear from the publicly available code from the Petitioner.

For the non-cached case, which include all paging I/O:

```
1136
1137         //
1138         // HANDLE THE NON-CACHED CASE
1139         //
1140
1141         if ( NonCachedIo ) {
1142
1143             ULONG SectorSize;
1144             ULONG BytesToRead;
1145
1146             DebugTrace(0, Dbg, "Non cached read.\n", 0);
1147
1148             //
1149             // Get the sector size
1150             //
1151
1152             SectorSize = (ULONG)Vcb->Bpb.BytesPerSector;
1153
1154             //
1155             // Start by zeroing any part of the read after Valid Data
1156             //
1157
1158             if (ValidDataLength < FcbOrDcb->ValidDataToDisk) {
1159
1160                 ValidDataLength = FcbOrDcb->ValidDataToDisk;
1161             }
1162
```

For the cached case, which the normal case for application initiated I/O:

```
1336
1337         //
1338         // HANDLE CACHED CASE
1339         //
1340
1341         else {
1342
1343             //
1344             // We delay setting up the file cache until now, in case the
1345             // caller never does any I/O to the file, and thus
1346             // FileObject->PrivateCacheMap == NULL.
1347             //
1348
```

In the cached case, the I/O is satisfied by invoking the Windows cache manager, which conceptually works like the system of Eylon: if the data is present in the cache, it is returned to the caller and if the data is not present in the cache, it is fetched from the storage location, which in the case of FastFat would be a local storage device:

DECLARATION OF WILLIAM ANTHONY MASON

Case IPR2021-0015

U.S. Patent No. 8,380,808

```
1383
1384         //
1385         // DO A NORMAL CACHED READ, if the MDL bit is not set,
1386         //
1387
1388         DebugTrace(0, Dbg, "Cached read.\n", 0);
1389
1390         if (!FlagOn(IrpContext->MinorFunction, IRP_MN_MDL)) {
1391
1392             //
1393             // Get hold of the user's buffer.
1394             //
1395
1396             SystemBuffer = FatMapUserBuffer( IrpContext, Irp );
1397
1398             //
1399             // Now try to do the copy.
1400             //
1401
1402             #if (NTDDI_VERSION >= NTDDI_WIN8)
1403                 if (!CcCopyReadEx( FileObject,
1404                                     &StartingByte,
1405                                     ByteCount,
1406                                     Wait,
1407                                     SystemBuffer,
1408                                     &Irp->IoStatus,
1409                                     Irp->Tail.Overlay.Thread )) {
1410             #else
1411                 if (!CcCopyRead( FileObject,
1412                                 &StartingByte,
1413                                 ByteCount,
1414                                 Wait,
1415                                 SystemBuffer,
1416                                 &Irp->IoStatus )) {
1417             #endif
1418
1419                 DebugTrace( 0, Dbg, "Cached Read could not wait\n", 0 );
```

In fact, the file systems can register handlers that are invoked by the Windows I/O manager so that the file system is not usually called to handle cached I/O operations at all. FastFat explicitly sets a standard Windows OS function to handle cached reads, using the “fast I/O” entry points:



```
174     DriverObject->FastIoDispatch = &FatFastIoDispatch;
175
176     RtlZeroMemory(&FatFastIoDispatch, sizeof(FatFastIoDispatch));
177
178     FatFastIoDispatch.SizeOfFastIoDispatch = sizeof(FAST_IO_DISPATCH);
179     FatFastIoDispatch.FastIoCheckIfPossible = FatFastIoCheckIfPossible; // CheckForFastIo
180     FatFastIoDispatch.FastIoRead = FsRtlCopyRead; // Read
181     FatFastIoDispatch.FastIoWrite = FsRtlCopyWrite; // Write
182     FatFastIoDispatch.FastIoQueryBasicInfo = FatFastQueryBasicInfo; // QueryBasicInfo
183     FatFastIoDispatch.FastIoQueryStandardInfo = FatFastQueryStdInfo; // QueryStandardInfo
184     FatFastIoDispatch.FastIoLock = FatFastLock; // Lock
185     FatFastIoDispatch.FastIoUnlockSingle = FatFastUnlockSingle; // UnlockSingle
186     FatFastIoDispatch.FastIoUnlockAll = FatFastUnlockAll; // UnlockAll
187     FatFastIoDispatch.FastIoUnlockAllByKey = FatFastUnlockAllByKey; // UnlockAllByKey
188     FatFastIoDispatch.FastIoQueryNetworkOpenInfo = FatFastQueryNetworkOpenInfo;
189     FatFastIoDispatch.AcquireForCcFlush = FatAcquireForCcFlush;
190     FatFastIoDispatch.ReleaseForCcFlush = FatReleaseForCcFlush;
191     FatFastIoDispatch.MdlRead = FsRtlMdlReadDev;
192     FatFastIoDispatch.MdlReadComplete = FsRtlMdlReadCompleteDev;
193     FatFastIoDispatch.PrepareMdlWrite = FsRtlPrepareMdlWriteDev;
194     FatFastIoDispatch.MdlWriteComplete = FsRtlMdlWriteCompleteDev;
```

Thus, in the normal case, application I/O does not reach the file system and instead triggers paging I/Os. This is consistent with the language of Eylon: “During normal operation...” Eylon does not benefit from intercepting these I/O operations. A POSITA would understand the distinction between cached and non-cached I/O operations and would ordinarily choose to handle only one, with non-cached I/O being the most common one to handle.

IV. Conclusion

The Petitioner's position that the '808 Patent is obvious over the prior art fails to bear up under scrutiny because:

- The Petitioner does not provide any cogent reason for a POSITA to combine Eylon with Christiansen. Eylon, by itself, is sufficient to provide streaming application delivery and Eylon would have been aware of file system filter drivers and chose not to use one for his implementation. Petitioner's position that Eylon includes a filter without using the term filter (Ex. 2134, Page 110 Line 23 to Page 111 Line 7) is unsupported. A file system, even a virtual file system, is not a filter and Petitioner offers no motivation to implement Eylon as a filter. However, Petitioner depends upon Eylon being a filter to motivate the combination with Christiansen to solve the very problems introduced by bludgeoning Eylon into a filter. Further, despite ample opportunity to do so, Petitioner fails to provide any motivation that is not tied to a filter (Ex. 2134, Page 106, Line 8 to Page 131). While I have separately explained why this combination might appear to be beneficial to a naïve POSITA, but in fact would not succeed (Ex. 2043, Page 72,) since Eylon does not teach using a file system filter to implement their system, there is no motivation to combine with Christiansen to resolve the ills that would have arisen *if* it had been implemented as a filter.
- The Petitioner relies upon the combination of Eylon and Christiansen to prove obviousness. Earlier I explained why there is no motivation to augment Eylon with Christiansen. Similarly, it is my opinion that there is no motivation to augment Christiansen with Eylon. When asked, the Petitioner could not name any specific feature lacking in Christiansen (Ex. 2134, Page 114 Line 9 to Page 122 Line 25.) In addition, as I have noted previously (Ex. 2043, Page 57) there is a conflict between Christiansen and Eylon with respect to handling of writes. That same conflict arises between Christiansen and the '808 Patent, which teaches that

changes to cached content are stored locally on the client computer (Ex. 2043, Page 34 and Ex. 1001Figure 4 and Ex. 1001Column 7 Line 61 to Column 8 Line 11) while Christiansen teaches they are written back to the remote storage location.

- The Petitioner takes a position that Eylon and Christiansen teach configurations that do not rely upon server elements. For Eylon, Petitioner claims that the server-side predictive streaming is an optional component of their system. I have observed that Eylon requires a server-based application delivery system (Ex. 2043, Page 41). Petitioner has argued that such a system *could* omit the server-initiated predictive streaming delivery (Ex. 1035, Paragraph 17). I have explained why removing that functionality would effectively turn Eylon's system into a "remote file server," which is the very thing that Eylon criticized (§ [A.](#)). However, even if the Board were to accept Petitioner's position with respect to server-side predictive streaming, it would not eliminate the server-side requirement that I previously described, which related to security and billing. Similarly, the Petitioner's position is that Christiansen could combine the cache server onto the client device (Ex. 2134, Page 130 Line 11 to Page 131 Line 6,) which would move the minifilter onto the client device. As I have noted, such a configuration defeats any data aggregation benefits from caching commonly accessed content between the storage location ("file server") and the consumer ("client") (§ [A.](#)). The '808 Patent has no server-side requirement.

While I have provided additional reasons why I disagree with the Petitioner's position, it is my opinion that these issues are the clearest in demonstrating that the various combinations of prior art that have been suggested by the Petitioner are insufficient to conclude that the claims of the '808 patent are obvious over the teachings of the prior art.

DECLARATION OF WILLIAM ANTHONY MASON

Case IPR2021-0015

U.S. Patent No. 8,380,808

Declaration

I, William Anthony Mason, do hereby declare and state that all statements made herein of my own knowledge are true and that all statements made on information and belief are believed to be true; and further that these statements were made with the knowledge that willful false statements and the like so made are punishable by fine or imprisonment, or both, under Section 1001 of Title 18 of the United States Code.

Dated: November 26, 2021

William Anthony Mason